

**LAPORAN WORKSHOP SISTEM INFORMASI
WEB SERVER
(TIF330805)**

SEMESTER III



BKPM: ACARA 4

**NAMA:
SHERYN FEBRYLIA HENDRITA
(E41250604)**

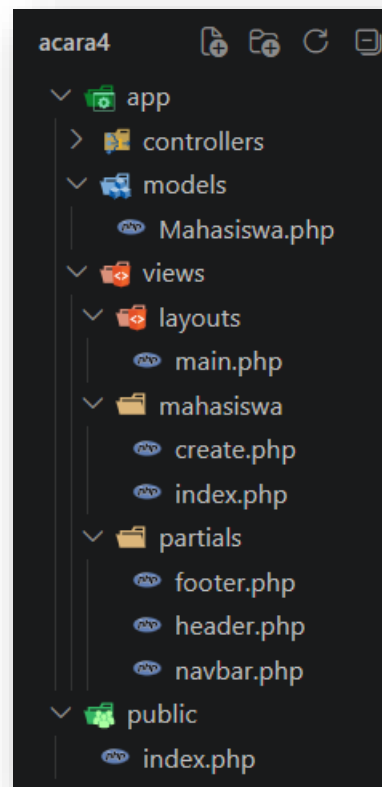
**PROGRAM STUDI DILUAR KAMPUS UTAMA (PSDKU)
TEKNIK INFORMATIKA KAB. NGANJUK
JURUSAN TEKNOLOGI INFORMASI
POLITEKNIK NEGERI JEMBER
TAHUN 2026**

A. Tujuan

Pada Acara 4 dilakukan pembuatan layout menggunakan partial, pembuatan class Mahasiswa sederhana dengan properti private dan getter, serta menampilkan data dari objek Mahasiswa ke dalam View.

Selain itu, pada tugas mandiri ditambahkan method `getAngkatan()` untuk menampilkan angkatan berdasarkan dua digit awal NIM.

B. Struktur Folder



1. partials/

Folder partials digunakan untuk menyimpan bagian-bagian kecil dari halaman yang dipakai berulang kali.

Contohnya:

```
partials/  
├── header.php
```

```
|— navbar.php
|— footer.php
```

Penjelasannya:

- header.php → berisi bagian awal HTML, seperti <html>, <head>, Bootstrap CDN, dan pembukaan <body>.
- navbar.php → berisi menu navigasi/header menu.
- footer.php → berisi bagian footer dan penutup HTML.

Jadi daripada setiap halaman menulis navbar sendiri, kita cukup memanggil:

```
require_once 'partials/navbar.php';
```

Intinya: partials = bagian-bagian halaman yang bisa digunakan kembali.

2. layouts/

Folder layouts digunakan untuk menyimpan kerangka/template halaman utama.

Misalnya nanti ada:

```
layouts/
|— main.php
```

Layout bisa menjadi tempat untuk mengatur struktur umum halaman, misalnya:

Header

↓

Navbar

↓

Isi halaman

↓

Footer

Jadi layout lebih seperti kerangka besar halaman, sedangkan partial adalah potongan-potongan bagian halaman.

C. Buat header.php

```
partials/header.php
```

```

app > views > partials > header.php
1  <!DOCTYPE html>
2  <html lang="id">
3
4  <head>
5      <meta charset="UTF-8">
6      <meta name="viewport" content="width=device-width, initial-scale=1.0">
7
8      <title><?= $title ?? 'Sistem Informasi Akademik' ?></title>
9
10     <link
11         href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css"
12         rel="stylesheet">
13 </head>
14
15 <body class="bg-light"></body>

```

D. Membuat navbar.php

partials/navbar.php

```

app > views > partials > navbar.php
1  <nav class="navbar navbar-dark bg-primary">
2      <div class="container">
3
4          <a href="#" class="navbar-brand">
5              Sistem Informasi Akademik
6          </a>
7
8      </div>
9  </nav>

```

E. Membuat footer.php

partials/footer.php

```
app > views > partials > footer.php
1  <footer class="bg-dark text-white text-center py-3 mt-5">
2      <p class="mb-0">
3          &copy; 2026 Sistem Informasi Akademik
4      </p>
5  </footer>
6
7  <script
8      src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/js/bootstrap.bundle.min.js">
9  </script>
10
11 </body>
12 </html>
```

Jadi kalau suatu saat footer berubah, kita cukup mengubah footer.php, tidak perlu mengubah setiap halaman. Itulah fungsi partial/include.

F. Membuat Class Mahasiswa

Sesuai soal, class dibuat dengan **property private + getter**.
app/Models/Mahasiswa.php

```

app > models > Mahasiswa.php > Mahasiswa
1  <?php
2
3  3 references | 0 implementations
4  class Mahasiswa
5  {
6      3 references
7      | private string $nim;
8      | 2 references
9      | private string $nama;
10     | 2 references
11     | private string $prodi;
12
13     3 references | 0 overrides
14     public function __construct(
15         string $nim,
16         string $nama,
17         string $prodi
18     ) {
19         $this->nim = $nim;
20         $this->nama = $nama;
21         $this->prodi = $prodi;
22     }
23
24     0 references | 0 overrides
25     public function getNim(): string
26     {
27         return $this->nim;
28     }
29
30     0 references | 0 overrides
31     public function getNama(): string
32     {
33         return $this->nama;
34     }
35
36     0 references | 0 overrides
37     public function getProdi(): string
38     {
39         return $this->prodi;
40     }
41
42     0 references | 0 overrides
43     public function getAngkatan(): string
44     {
45         return '20' . substr($this->nim, 0, 2);
46     }
47 }

```

```

29     public function getProdi(): string
30     {
31         return $this->prodi;
32     }
33
34     0 references | 0 overrides
35     public function getAngkatan(): string
36     {
37         return '20' . substr($this->nim, 0, 2);
38     }
39 }

```

Kenapa private?

Karena data mahasiswa tidak diakses langsung dari luar class.

Untuk mengambil datanya digunakan getter:

```
$mhs->getNim();  
$mhs->getNama();  
$mhs->getProdi();  
$mhs->getAngkatan();
```

G. Membuat Object di index.php

Pada file:

public/index.php

Dibuat beberapa object mahasiswa:

```
public > index.php  
1  <?php  
2  
3  require_once __DIR__ . '/../app/models/Mahasiswa.php';  
4  
5  $mahasiswa = [  
6      new Mahasiswa(  
7          '25001',  
8          'Sheryn Febrylia Hendrita',  
9          'Teknik Informatika'  
10     ),  
11  
12     new Mahasiswa(  
13         '25002',  
14         'Ingka Jivanda Gayshela',  
15         'Teknik Informatika'  
16     ),  
17  
18     new Mahasiswa(  
19         '25003',  
20         'Dinda Febiola Rachmawati',  
21         'Teknik Informatika'  
22     )  
23 ];  
24  
25 $title = 'Data Mahasiswa';  
26  
27 ob_start();  
28  
29 require __DIR__ . '/../app/views/mahasiswa/index.php';  
30  
31 $content = ob_get_clean();  
32  
33 require __DIR__ . '/../app/views/layouts/main.php';
```

Data tersebut kemudian dikirim ke View.

H. Memanggil View Mahasiswa

Pada public/index.php, View dipanggil menggunakan:

```
27  ob_start();
28
29  require __DIR__ . '/../app/views/mahasiswa/index.php';
30
31  $content = ob_get_clean();
32
33  require __DIR__ . '/../app/views/layouts/main.php';
```

Penjelasan:

ob_start() digunakan untuk mulai menangkap output dari View.

Kemudian:

```
require __DIR__ . '/../app/views/mahasiswa/index.php';
```

Digunakan untuk menjalankan View mahasiswa.

Hasilnya ditampung ke:

```
$content = ob_get_clean();
```

Kemudian \$content digunakan pada layout utama main.php.

I. Menampilkan Data pada View

File:

app/views/mahasiswa/index.php

digunakan untuk menampilkan data dalam tabel Bootstrap.

Bagian tabel memiliki kolom:

```

17 <div class="card shadow-sm border-0">
18
19 <div class="card-body">
20
21 <div class="table-responsive">
22
23 <table class="table table-bordered table-hover align-middle mb-0">
24
25 <thead class="table-primary">
26
27 <tr>
28 <th>No</th>
29 <th>NIM</th>
30 <th>Nama</th>
31 <th>Program Studi</th>
32 <th>Angkatan</th>
33 </tr>
34
35 </thead>
36

```

Kemudian data ditampilkan menggunakan foreach:

```

37 <tbody>
38
39 <?php foreach ($mahasiswa as $index => $mhs): ?>
40
41 <tr>
42
43 <td>
44 <?= $index + 1 ?>
45 </td>
46
47 <td>
48 <?= htmlspecialchars($mhs->getNim()) ?>
49 </td>
50
51 <td>
52 <?= htmlspecialchars($mhs->getNama()) ?>
53 </td>
54
55 <td>
56 <?= htmlspecialchars($mhs->getProdi()) ?>
57 </td>
58
59 <td>
60 <?= htmlspecialchars($mhs->getAngkatan()) ?>
61 </td>
62
63 </tr>
64
65 <?php endforeach; ?>
66
67 </tbody>

```

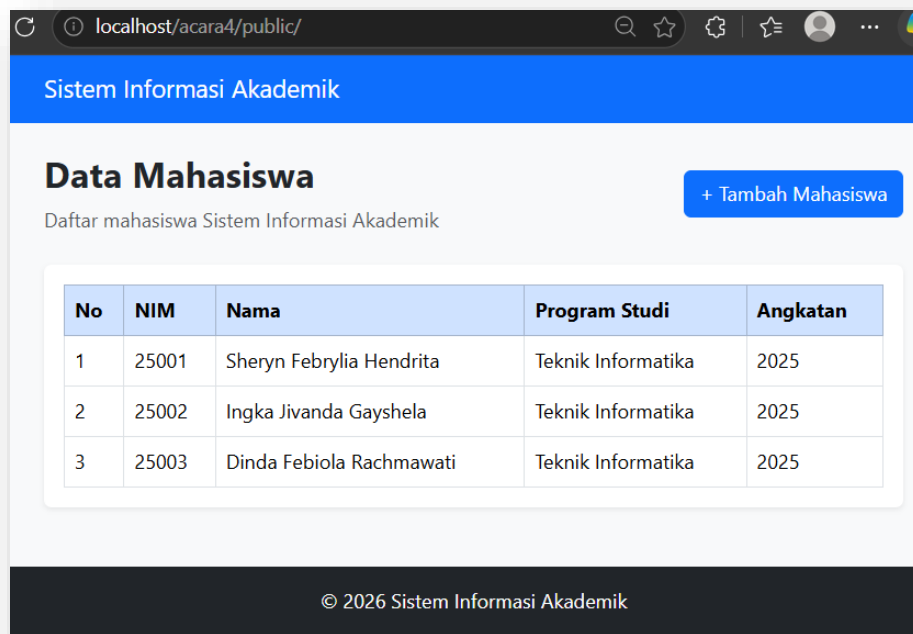
Dengan demikian, View tidak membuat object baru. View hanya menampilkan object yang sudah dibuat di public/index.php.

J. Menjalankan Program

Pastikan Laragon sudah dijalankan.

Kemudian buka browser dan akses:

<http://localhost/acara4/public/>



K. Pembahasan

Pada Acara 4 diterapkan konsep **partial**, **layout**, **class**, **object**, dan **View**. Bagian halaman yang digunakan secara berulang dipisahkan ke dalam file partial sehingga kode tidak perlu ditulis kembali pada setiap halaman.

Class Mahasiswa digunakan untuk merepresentasikan data mahasiswa dengan property private, yaitu NIM, nama, dan program studi. Data tersebut diakses melalui method getter.

Pada `public/index.php`, dibuat beberapa object Mahasiswa menggunakan data mahasiswa yang telah ditentukan. Object tersebut kemudian dikirim ke View mahasiswa/`index.php` untuk ditampilkan dalam bentuk tabel.

Pada tugas mandiri, ditambahkan method `getAngkatan()` yang menggunakan `substr()` untuk mengambil dua digit awal NIM. Pada data yang digunakan, NIM 25001, 25002, dan 25003 menghasilkan angkatan 25.

Penggunaan layout dan partial membuat struktur aplikasi lebih terorganisir karena bagian yang bersifat umum dapat dipisahkan dari bagian yang menampilkan data mahasiswa.

L. Kesimpulan

Berdasarkan Acara 4, telah berhasil diterapkan konsep partial dan layout serta pembuatan class Mahasiswa dengan property private dan getter. Beberapa object mahasiswa berhasil dibuat pada `public/index.php` dan ditampilkan melalui View dalam bentuk tabel Bootstrap.

Selain itu, method `getAngkatan()` berhasil ditambahkan untuk mengambil dua digit awal NIM sehingga angkatan mahasiswa dapat ditampilkan pada tabel. Dengan struktur tersebut, kode menjadi lebih terorganisir dan bagian tampilan dapat dipisahkan sesuai fungsinya.